



3D点云pointnet++学习随手记

PointNet++要解决的问题：

PointNet++的核心目标，就是在PointNet的基础上，引入提取局部特征的能力，并构建一个层次化的特征学习框架。这就像CNN在处理图像时，通过多层卷积从边缘、纹理到物体部件，逐步提取越来越复杂的特征一样，PointNet++也希望在点云上实现类似的效果。

▼ CNN:卷积神经网络

一个典型的CNN识别图像的流程可以概括为：

1. 输入：原始图像（像素矩阵）。
2. 特征提取阶段：
 - 第一层卷积：探测低级特征，如边缘、线条、色块。
 - 第一层池化：压缩特征图，增强鲁棒性。
 - 第二层卷积：基于低级特征，组合成更复杂的特征，如眼睛、鼻子、轮子。
 - 第二层池化：再次压缩。
 - （重复多次）：更深层的卷积会探测到越来越抽象和全局的特征，如“脸”、“车身”。
3. 分类决策阶段：

- **展平**：将所有高级特征图转换成一维向量。
- **全连接层**：学习这些特征之间的复杂组合关系。
- **输出层**：输出每个类别的概率，概率最高的即为预测结果

核心思想：层次化特征学习

PointNet++的基本思想很直观：递归地在点云上应用一个局部特征学习模块，这个模块就是一个小型的PointNet。通过这种方式，网络能够从小范围局部区域逐步扩展到更大范围，最终捕获整个点云的全局特征。

网络架构与关键技术

Set Abstraction（集合抽象）

这是PointNet++的核心模块，由三个阶段构成：

采样（Sampling）：

使用**最远点采样（Farthest Point Sampling, FPS）** 算法从输入点中选取一组中心点。FPS的好处是，在给定相同数量中心点的情况下，它能比随机采样**更均匀地覆盖整个点云**。

▼ 1. 采样（Sampling）：如何选出“有代表性”的中心点？

目标：从输入点云（例如N个点）中，选出M个点（ $M < N$ ）作为局部区域的中心。

方法：最远点采样（Farthest Point Sampling, FPS）

原理与步骤：

FPS的核心思想是**最大化覆盖**，即选出的点应尽可能地均匀分散在整个点云中，而不是挤在一起。

1. **随机开局**：从所有点中随机选择一个点作为第一个中心点。
2. **距离计算**：计算所有其他点到**当前已选中心点集合的最短距离**。也就是说，对于每一个点，它需要找出离它最近的那个中心点，并记录这个距离。
3. **选择最远者**：在所有点的这些最短距离中，选择**距离最大的那个点**。这个点就是离当前所有中心点“最远”的点，将它加入中心点集合。
4. **循环往复**：重复步骤2和3，直到选出了预定数量（M）的中心点。

举例说明：

假设我们有一个简单的2D点集，包含5个点：A, B, C, D, E。我们要用FPS选出3个中心点。

1. **随机选择A**作为第一个中心点。
2. **第一轮计算**：计算B, C, D, E到A的距离。假设结果是：B:3, C:5, D:4, E:6。那么，E点（距离6）是离A最远的点。
3. **选择E**作为第二个中心点。现在中心点集合是 {A, E}。
4. **第二轮计算**：计算每个点到 {A, E} 这个集合的最短距离。
 - B: 到A=3， 到E=5， 最短距离是3。
 - C: 到A=5， 到E=4， 最短距离是4。
 - D: 到A=4， 到E=3， 最短距离是3。
 - （A和E本身已选，忽略）
 - 这些最短距离中，C点的4是最大的。
5. **选择C**作为第三个中心点。最终选出的中心点是 {A, E, C}。

你可以看到，FPS选出的点分布非常均匀，有效地覆盖了整个空间。如果使用随机采样，很可能会选中两个距离很近的点，导致局部区域重叠，信息冗余。

分组 (Grouping):

围绕每个采样中心点，构建局部区域。常用的是 **Ball Query** 方法，即选取以中心点为球心、在**指定半径内**的所有点。这确保了局部区域具有**固定的空间尺度**。每个区域内的点数可能不同，但通过后续的PointNet可以处理。

▼ 2. 分组 (Grouping): 如何构建局部区域?

目标：为每一个上一步选出的中心点，划定一个“邻居范围”，构建一个局部点集。

方法：Ball Query (球查询)

原理：

为每个中心点，在三维空间中划定一个半径为R的球体。所有落在这个球体内的点（包括中心点自己）都被认为是该中心点的“邻居”，共同构成一个局部区域。

关键特性：

- **固定空间尺度**：半径R是固定的。这意味着无论点云密度如何，每个局部区域在物理空间中的大小是固定的。
- **点数不固定**：由于点云密度可能不均，不同中心点球体内的点数（K）可能不同。有的区域点密集，K就大；有的稀疏，K就小。

举例说明：

假设我们有一个室内场景的点云。我们用FPS选出了几个中心点，包括一个在桌子腿上的点P1和一个在远处墙壁上的点P2。现在我们用Ball Query（半径R=0.2米）进行分组。

- 对于**P1（桌子腿）**，周围结构复杂，在半径为0.2米的球内可能包含了桌子腿的多个侧面点，假设有**50个点**。
- 对于**P2（墙壁）**，墙壁表面相对平坦，在同样0.2米的球内可能只包含了墙壁表面的一些点，假设只有**15个点**。

这样，我们就为P1和P2构建了大小相同但内部点数不同的局部区域。网络需要能够处理这种**可变长度**的输入。

特征提取（Feature Extraction）：

对每个分组好的局部区域，使用一个**小型PointNet**进行特征提取。这个过程会将**整个局部区域的信息聚合（例如通过最大池化）**成一个代表该区域的高级特征向量，并赋予该中心点。

通过堆叠多个Set Abstraction层，点云的点数逐步减少（采样），而每个点的特征维度不断增加，感受野也越来越大，从而实现了**从局部到全局的层次化特征学习**。

▼ 3. 特征提取：如何从局部区域中提炼出特征？

目标：将上一步得到的、大小不一的局部点集，转换成一个固定长度的、有代表性的特征向量。

方法：微型PointNet（Mini-PointNet）

原理与步骤：

这是PointNet思想的直接应用。每个局部区域都被视为一个独立的“小点云”，送入一个共享权重的多层感知机（MLP）和对称函数（最大池化）中进行处理。

1. **输入**：一个局部区域有K个点，每个点有 $d + c$ 维特征。 d 是坐标维度（如 x, y, z ）， c 是额外的特征维度（如颜色、法向量，或上一层的特征）。

2. **通过共享MLP**：每个点独立地通过一个共享的MLP。这个MLP的作用是**学习每个点的逐点特征**，并将其映射到更高维的特征空间。
3. **最大池化**：对所有K个点经过MLP后的特征，在点的维度（K）上进行**最大池化（Max Pooling）**。这一步是**聚合局部信息**的关键，它从所有点的特征中提取出最显著的信息，形成一个**对该局部区域具有“判别性”的特征向量**。
4. **输出**：得到一个**固定长度**的特征向量，作为该中心点的新特征。这个新特征编码了整个局部区域的几何结构信息。

举例说明：

假设一个局部区域描述了一个“桌角”。

- **输入**：这个区域有K=30个点，每个点有坐标(x,y,z)和来自上一层的64维特征。所以输入是 $(30, 3+64)$ 的张量。
- **共享MLP**：一个共享的MLP（例如 $[67 \rightarrow 64 \rightarrow 128]$ ）被应用到这30个点上。输出是 $(30, 128)$ 的张量。现在每个点都有一个128维的特征，这个特征描述了它自身及其在局部上下文中的角色。
- **最大池化**：沿着30个点这个维度，对128个特征通道的每一个，取最大值。输出是一个 $(1, 128)$ 的特征向量。
- **结果**：这个128维的向量，就是整个“桌角”这个局部结构的抽象表示。它捕捉到了这个区域是“一个角落”而不是一个“平面”或“边缘”的关键信息

层次化堆叠：构建特征金字塔

通过堆叠多个Set Abstraction层，网络构建了一个层次化的特征学习过程：

- **第一层**：输入可能是1024个点。Set Abstraction采样出512个中心点，每个点的局部区域很小（半径R1小），学习到的是**非常局部的特征**，比如点、边、小平面。
- **第二层**：以第一层的512个点及其特征作为输入。再采样出128个中心点。此时，每个中心点的局部区域（半径R2 > R1）覆盖了第一层的多个局部区域。因此，它学习到的是**更复杂的部件级特征**，比如桌腿、桌面。
- **第三层**：进一步采样出更少的点（如32个）。感受野变得更大，最终通过全局池化，学习到**整个物体的全局特征**，比如“这是一张桌子”。

PointNet++ 中两种不同的局部特征提取策略

SSG vs MSG

SSG (Single-Scale Grouping) - 单尺度分组

工作原理：

- 在每个层级使用**固定大小**的邻域来分组点
- 类似于使用**固定倍率**的放大镜观察点云

```
# 简化理解
SSG_process = {
    "层级1": "使用半径R1的球邻域",
    "层级2": "使用半径R2的球邻域",
    "层级3": "使用半径R3的球邻域",
    "特点": "每个层级尺度固定"
}
```

SSG实现方式

```
def SSG_grouping(points, radius):
    # 1. 对每个点，在固定半径内找邻居
    # 2. 提取局部特征
    # 3. 传递到下一层
    return local_features
```

MSG (Multi-Scale Grouping) - 多尺度分组

工作原理：

- 在每个层级**同时使用多个不同大小**的邻域
- 类似于**同时使用多个倍率**的放大镜观察同一个区域

```
# 简化理解
MSG_process = {
    "层级1": "同时使用半径[R1, R2, R3]的球邻域",
    "层级2": "同时使用半径[R4, R5, R6]的球邻域",
    "特点": "每个层级多尺度并行"
}
```

MSG实现方式

```
def MSG_grouping(points, radii_list):
    features = []
    for radius in radii_list: # [0.1, 0.2, 0.4]
        # 对不同半径分别处理
        local_feat = group_by_radius(points, radius)
        features.append(local_feat)

    # 融合多尺度特征
    return combine_features(features)``python
def MSG_grouping(points, radii_list):
    features = []
    for radius in radii_list: # [0.1, 0.2, 0.4]
        # 对不同半径分别处理
        local_feat = group_by_radius(points, radius)
        features.append(local_feat)

    # 融合多尺度特征
    return combine_features(features)
```

总结：SSG就像：

 使用单一焦距的相机

- 优点：简单、快速、省电
- 缺点：可能错过某些尺度的细节

MSG就像：

 使用多镜头相机系统

- 广角镜头：捕捉大范围上下文
- 标准镜头：平衡的视角
- 长焦镜头：捕捉精细细节
- 优点：全面、鲁棒、精度高
- 缺点：更重、更贵、更耗电

▼ 测试结果

MSG:Epoch 15/15:
Train Loss: 0.3652
Train Accuracy: 88.58%
Time: 296.54s
Test Accuracy: 87.97%

SSG:Epoch 10/10:
Train Loss: 0.4551
Train Accuracy: 85.46%
Time: 188.27s
Test Accuracy: 86.47%

应对点云密度不均问题：MSG与MRG

真实场景中点云的密度往往不均匀（例如，离传感器近的地方点密，远的地方点疏）。这会导致在固定半径内，不同区域的点数量差异巨大，影响特征提取的稳定性。为此，PointNet++提出了两种策略：

▼ 多尺度分组（Multi-Scale Grouping, MSG）

1. 多尺度分组：

对于同一个中心点，不是只用一个半径 R 进行Ball Query，而是使用**一组半径** $[R_1, R_2, R_3, \dots]$ （例如 $R_1=0.1$, $R_2=0.2$, $R_3=0.4$ ）分别进行分组。这样就得到了同一个中心点的**多个不同空间尺度**的局部区域。

2. 并行特征提取：

每个尺度的局部区域都独立地通过一个**专用的PointNet**（每个尺度有自己的MLP权重，不共享）进行特征提取。每个PointNet输出一个特征向量。

3. 特征拼接：

将所有不同尺度提取出的特征向量**拼接（Concatenate）**在一起，形成该中心点的最终特征。

核心思想：与其猜测哪个尺度最合适，不如**同时观察所有可能的尺度**，让网络自己学习如何权衡。

对**同一个中心点**，使用**多个不同半径**进行分组，每个半径尺度都通过一个PointNet提取特征，然后将这些**不同尺度的特征拼接起来**作为该中心点的**最终特征**。这种方式能捕获更丰富的上下文信息，但**计算量较大**。

▼ 多分辨率分组（Multi-Resolution Grouping, MRG）：

详细原理

MRG特征由**两部分**拼接而成：

1. 局部特征 (Local Feature)：

- **来源**：通过**当前层**的Set Abstraction提取的特征。即对当前层分组得到的局部区域应用PointNet后得到的特征。
- **特点**：这个特征基于**当前采样后相对稀疏的点集**，但它经过了更深层的网络处理，理论上**语义信息更高级**。

2. 全局特征 (Global Feature, 或称Fine-grained Feature)：

- **来源**：直接对**前一层的所有原始点**，以其与中心点的距离为权重，应用一个**独立的PointNet**提取的特征。具体来说，是在前一层的点中，选取距离当前中心点最近的那些点（例如K近邻），然后用PointNet处理。
- **特点**：这个特征基于**更底层、更密集的原始点云**，因此它保留了更多的几何细节和精细信息。

自适应密度的工作机制

MRG的巧妙之处在于其**自适应能力**：

- **高密度区域**：当前层的局部区域包含足够多的点，提取的**局部特征非常可靠和精细**。此时，网络可以**主要依赖局部特征**，而全局特征作为补充。
- **低密度区域**：当前层的局部区域可能只有寥寥几个点，提取的局部特征**噪声大、不可靠**。此时，来自前一层的**全局特征就显得至关重要**，因为它基于更多、更密集的点计算而来，提供了更稳定、更可靠的特征表示。

可以类比为：局部特征是一个“专家”，在数据充足时判断精准；全局特征是一个“经验丰富的老兵”，在信息不足时能提供稳健的估计。MRG让网络自己决定在何时更相信谁。

核心思想：用一种更聪明、更高效的方式来融合多尺度信息。不直接使用多个物理尺度，而是利用网络**不同深度（层次）的特征**来隐含地提供多尺度信息。

为了平衡性能与计算量，MRG被提出。如所述，MRG特征由两部分组成：

- 来自**当前层**的局部区域特征（通过Set Abstraction提取）。
- 来自**前一层的对应区域**，直接对所有原始点使用PointNet提取的特征。

当局部区域点密度较低时，当前层的特征可能不可靠（因为点太少），此时可以更多地依赖来自前一层的、基于更多原始点计算的特征；当密度高时，则更信赖当前层提取的精细特征。MRG让网络能自适应地点云的密度变化。

分类与分割任务实现

1. 分类网络 (Classification)

分类网络的结构相对直接，它是一个**编码器 (Encoder)** 结构：

多个**Set Abstraction**层 → 提取出**全局特征** → 通过**全连接层 (MLP)** → 输出**分类得分**。

2. 分割网络 (Segmentation)

分割任务需要为每个点预测一个类别标签，因此是**逐点 (point-wise)** 的输出。

PointNet++采用了一种**编码器-解码器 (Encoder-Decoder)** 结构：

- **编码器**：与分类网络相同，通过Set Abstraction下采样并提取特征。
- **解码器**：通过**特征传播 (Feature Propagation)** 层进行**上采样**，逐步恢复点的数量并融合不同层级的特征。
 - **插值 (Interpolation)**：通过**反距离加权插值**等方法，将高层稀疏点的特征传播到低层更密集的点上。
 - **跳跃连接 (Skip Connection)**：将插值后的特征与编码器中对应层（通过跳跃连接而来）的特征进行**拼接 (Concatenate)**。这使得解码过程中的点特征同时包含了**详细的局部信息 (来自浅层)** 和**高级的语义信息 (来自深层)**。
 - 最后，再通过一个**共享的MLP (Unit PointNet)** 对每个融合后的特征进行处理，输出每个点的分类结果。

下面的表格总结了这两种网络的特点：

任务	网络结构	核心目标	输出特点
分类	编码器 (Encoder)	提取 全局特征	单个结果代表整个点云的类别
分割	编码器-解码器 (Encoder-Decoder)	融合 局部与全局特征 ，实现 逐点分类	输出序列，每个点都有其类别标签

总结、评价与应用

- **核心贡献**：PointNet++通过引入**层次化的Set Abstraction**结构和**多尺度特征融合策略 (MSG/MRG)**，成功解决了PointNet难以捕捉局部特征和对点云密度变化敏感的问题。

- **性能：**在多个标准数据集上，PointNet++在分类和分割任务上都达到了超越PointNet和当时其他方法的性能，尤其是在复杂场景和稀疏点云上表现出更好的鲁棒性。
- **应用：**PointNet++为后续的点云深度学习研究奠定了重要基础，其思想被广泛应用，例如在**自动驾驶**（LiDAR点云处理）、**三维重建**、**机器人导航**等领域。一些后续研究也通过在PointNet++基础上引入注意力机制等方式进一步提升性能。